

API SECURITY RISK ANALYSIS REPORT

Shrilee Shah
Cybersecurity Intern
Future Interns

CIN: FIT/MAY26/CS8385

Date: 05/06/2026

Table Of Contents

1. Executive Summary
2. Objective
3. Scope of Assessment
4. Methodology
5. Risk Rating Methodology
6. Findings
 - a. Unauthenticated Access to API Resources
 - b. Excessive Data Exposure
 - c. Predictable Object Access (ID Enumeration Risk)
 - d. Missing Rate Limiting Indicators
 - e. Limited Security Headers
7. Summary Risk Matrix
8. Testing Evidence
9. Remediation Suggestions
10. Conclusion

Executive Summary

Application Programming Interfaces (APIs) form the backbone of modern web applications, mobile applications, Software-as-a-Service (SaaS) platforms, cloud services, and third-party integrations. Because APIs often process and expose sensitive information, they are a common target for cyber attackers.

This assessment was conducted to evaluate the security posture of a publicly available demonstration API, JSONPlaceholder, through a read-only security review. The objective was not to exploit vulnerabilities but to identify security risks that would be significant if observed within a production environment.

The assessment focused on authentication mechanisms, authorization controls, data exposure, response behaviour, HTTP headers, and rate-limiting indicators. Testing was performed using Postman and publicly available API documentation.

Several observations were identified including unrestricted access to endpoints, exposure of user-related information, predictable object identifiers, absence of visible rate-limiting controls, and limited security-related response headers. Although the target API is intentionally designed for testing and learning purposes, these findings demonstrate common API security weaknesses that organizations should address in production systems.

Objective

- Review publicly accessible API endpoints
- Analyse authentication requirements
- Examine authorization controls
- Evaluate exposed response data
- Review HTTP response headers
- Identify potential security risks
- Assess business impact
- Recommend remediation measures

Scope of assessment

Target API:

<https://jsonplaceholder.typicode.com>

Assessment Type:

Read-Only API Security Risk Analysis

Testing Method:

Manual inspection using Postman

In Scope:

- Endpoint review
- Response inspection
- Authentication analysis
- Authorization observations
- Security header review

Out of Scope:

- Exploitation attempts
- Denial of Service testing
- Vulnerability exploitation
- Attacks against production systems

Methodology

The following methodology was used during the assessment:

Phase 1 – Documentation Review

Public API documentation was reviewed to understand available endpoints, request methods, expected responses, and intended functionality.

Phase 2 – Endpoint Enumeration

Several publicly available endpoints were tested to determine accessibility and response behaviour.

Examples:

GET /users

GET /posts

GET /users/1

Phase 3 – Response Analysis

Returned data was reviewed to identify potentially sensitive information and unnecessary data exposure.

Phase 4 – Header Inspection

HTTP response headers were inspected to identify security-related configurations and indicators of defensive controls.

Phase 5 – Risk Assessment

Observed findings were categorized according to their potential impact on confidentiality, integrity, and availability.

Risk Rating Methodology

Severity ratings were assigned based on:

- Likelihood of exploitation
- Potential business impact
- Potential data exposure
- Potential operational impact

Risk Levels:

- High – Significant business impact and likely exploitation
- Medium – Moderate business impact requiring remediation
- Low – Limited impact but recommended improvement

Finding 1 – Unauthenticated Access to API Resources

1. Severity: Medium

2. Description

API endpoints were accessible without requiring authentication credentials, API keys, access tokens, or user login mechanisms.

3. Observation

Requests to user-related and content-related endpoints successfully returned data despite the absence of any authentication mechanism.

4. Potential Business Impact

In production systems, unrestricted access could expose confidential customer data, internal records, or business information. Unauthorized access may lead to compliance violations and reputational damage.

5. Recommendation

Implement strong authentication mechanisms such as OAuth 2.0, OpenID Connect, API keys, or JWT-based authentication. Access should be restricted according to business requirements.

Finding 2 – Excessive Data Exposure

1. Severity: Medium

2. Description

The API returned multiple user attributes including names, email addresses, phone numbers, company details, website information, and location-related data.

3. Observation

Responses contained more information than may be necessary for typical user-facing operations.

4. Potential Business Impact

Excessive data exposure increases privacy risks and provides attackers with valuable information for reconnaissance, phishing campaigns, and social engineering attacks.

5. Recommendation

Apply the principle of data minimization. Return only fields required by the requesting application or user.

Finding 3 – Predictable Object Access (ID Enumeration Risk)

1. Severity: Medium

2. Description

Resources were accessible using sequential identifiers such as /users/1.

3. Observation

Object identifiers appeared predictable and directly accessible.

4. Potential Business Impact

In real-world environments, attackers may enumerate identifiers to access information belonging to other users if authorization checks are weak or missing.

5. Recommendation

Implement object-level authorization controls and validate user permissions before serving requested resources.

Finding 4 – Missing Rate Limiting Indicators

1. Severity: Medium

2. Description

No visible evidence of rate limiting controls was observed during testing.

3. Observation

Response headers did not clearly indicate throttling policies or request limitations.

4. Potential Business Impact

Attackers could automate requests to scrape data, perform credential attacks, or exhaust application resources.

5. Recommendation

Implement rate-limiting mechanisms, request throttling, API gateways, and monitoring controls.

Finding 5 – Limited Security Headers

1. Severity: Low

2. Description

Only basic HTTP response headers were observed during testing.

3. Observation

Common security headers were not prominently visible.

4. Potential Business Impact

Security headers help reduce attack surface and improve application hardening.

5. Recommendation

Review API gateway configuration and implement appropriate security headers wherever applicable.

Summary Risk Matrix

Finding	Severity
Unauthenticated Access	Medium
Excessive Data Exposure	Medium
ID Enumeration Risk	Medium
Missing Rate Limiting	Medium
Limited Security Headers	Low

Testing Evidence

GET Get data

GET Get data

My Collection - Get data

GET - https://jsonplaceholder.typicode.com/users

Send

Docs Params Authorization Headers Body Scripts Tests Settings

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers Test Results 1/1

200 OK · 879 ms · 1.23 KB · Save Response

JSON Preview Visualize

```
1 {
2   "id": 1,
3   "name": "Leanne Graham",
4   "username": "Bret",
5   "email": "Sincere@april.biz",
6   "address": {
7     "street": "Kulas Light",
8     "suite": "Apt. 556",
9     "city": "Gwenborough",
10    "zipcode": "92998-3874",
11    "geo": {
12      "lat": "37.3159",
13      "lng": "81.1496"
14    }
15  },
16  "phone": "1-770-736-8031 x56442",
17  "website": "hildegard.org",
18  "company": {
19    "name": "Romaguera-Crona",
20    "catchPhrase": "Multi-layered client-server neural-net",
21    "bs": "harness real-time e-markets"
22  }
23 }
```

GET Get data

GET Get data

My Collection - Get data

GET - https://jsonplaceholder.typicode.com/users/1

Send

Docs Params Authorization Headers Body Scripts Tests Settings

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers Test Results 1/1

200 OK · 879 ms · 1.4 KB · Save Response

JSON Preview Visualize

```
1 {
2   "id": 1,
3   "name": "Leanne Graham",
4   "username": "Bret",
5   "email": "Sincere@april.biz",
6   "address": {
7     "street": "Kulas Light",
8     "suite": "Apt. 556",
9     "city": "Gwenborough",
10    "zipcode": "92998-3874",
11    "geo": {
12      "lat": "37.3159",
13      "lng": "81.1496"
14    }
15  },
16  "phone": "1-770-736-8031 x56442",
17  "website": "hildegard.org",
18  "company": {
19    "name": "Romaguera-Crona",
20    "catchPhrase": "Multi-layered client-server neural-net",
21    "bs": "harness real-time e-markets"
22  }
23 }
```

GET Get data

GET Get data

No environment

My Collection > Get data

Save

Share

GET

https://jsonplaceholder.typicode.com/users

Send

Docs

Params

Authorization

Headers 0

Body

Scripts

Tests

Settings

Cookies

headers

6 hidden

Key	Value	Description
Key	Value	Description

Body

Cookies

Headers 26

Test Results 1/1

200 OK

347 ms

2.93 KB

Save Response

Key	Value
status	200
date	Thu, 04 Jun 2026 17:58:08 GMT
content-type	application/json; charset=utf-8
content-length	1847
access-control-allow-credentials	true
cache-control	max-age=43200
content-encoding	gzip
etag	W/"160d-1eMSsxeJRfnVLRBmYJSbCUZ1qQ"
expires	-1
nel	{"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01...
pragma	no-cache
report-to	{"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=oggjGsfe0Ff1MAX...
reporting-endpoints	heroku-nel="https://nel.heroku.com/reports?s=oggjGsfe0Ff1MAXtigC%2FDYoOSKo%2FaQGp...
server	cloudflare
vary	Origin, Accept-Encoding

Globals

Vault

Tools

Remediation Suggestions

1. Implement Strong Authentication

Require authentication mechanisms such as OAuth 2.0, JWT tokens, or API keys before allowing access to sensitive endpoints. This helps prevent unauthorized access to API resources.

2. Enforce Proper Authorization

Implement role-based access control (RBAC) and object-level authorization checks to ensure users can only access data and resources they are permitted to view.

3. Minimize Data Exposure

Return only the information required for the intended operation. Avoid exposing unnecessary user details such as contact information, addresses, or internal identifiers.

4. Apply Rate Limiting

Introduce request throttling and rate-limiting controls to protect APIs from abuse, automated attacks, and excessive resource consumption.

5. Strengthen Security Configurations

Use appropriate security headers, secure API gateway configurations, and HTTPS encryption to improve the overall security posture of the API.

6. Enable Monitoring and Logging

Maintain detailed logs of API activity and implement monitoring mechanisms to detect suspicious behaviour, unauthorized access attempts, and potential security incidents.

7. Conduct Regular Security Assessments

Perform periodic vulnerability assessments, security reviews, and API testing to identify and remediate security weaknesses before they can be exploited.

Conclusion

The API Security Risk Analysis demonstrated a structured approach for evaluating modern APIs using safe and ethical testing methods. Although JSONPlaceholder is intentionally designed as a public testing platform, the identified observations represent common security concerns frequently encountered during API security reviews.

Organizations should implement robust authentication, strong authorization controls, data minimization practices, rate limiting, monitoring, and secure configuration management to protect APIs against evolving threats.

API security should be treated as a continuous process integrated throughout the software development lifecycle to ensure long-term protection of business assets and customer information.